本科毕业设计开题报告

题目： 命题式类型约束：将类型约束提升为逻辑推理

学　　院： 计算机科学与信息安全学院

专　　业： 软件工程/计算机科学

姓　　名： 黄荣钊

学　　号： 2200310717

指导教师： 肖雁南

2025 年 9 月 30 日

1 毕业设计的内容

1.1 引言与研究背景

1.1.1 静态分析在系统编程中的核心困境

在现代系统编程语言的设计中，静态分析能力是保证软件可靠性、安全性和性能的关键。然而，语言设计者普遍面临一个棘手的“不可能三角”困境，即需要在以下三个目标之间进行权衡：

- **表达力 (Expressiveness)**: 语言的类型系统能够静态地描述和检查多复杂的程序属性与不变量。
- **定义期可检查性 (Definition-Site Checkability)**: 在泛型代码被具体实例化之前，编译器能在多大程度上验证其定义的自洽性与正确性。
- **决策复杂性 (Decidability & Complexity)**: 类型检查和约束求解的计算复杂性是否可控，以及编译速度是否可以接受。

这个困境导致了当前主流系统语言在设计哲学上的显著分化。

1.1.2 现有技术方案及其局限性

对现有典型语言的考察揭示了它们在上述困境中的不同取舍：

- **Zig 语言的 `comptime` 机制**选择了极致的表达力。它的编译期执行能力是图灵完备的，允许开发者实现任意复杂的编译期逻辑。然而，这种能力的代价是牺牲了定义期的可检查性。泛型代码的正确性只有在被具体类型实例化并“运行”后才能得到验证，这常常导致错误信息出现在远离问题根源的调用点，且错误信息晦涩难懂，被社区戏称为“编译时的运行时错误”。

- **Rust 语言的 Trait 系统**则将可检查性和一致性置于首位。其类型系统基于强大的霍恩子句逻辑，能够在定义端提供强有力的静态保证，并产生高质量的错误诊断。然而，这种严谨性也限制了其表达力。Rust 的 Trait 系统原生不支持逻辑析取 (`or`) 或否定 (`not`)，使得某些在逻辑上合理的约束（例如“类型 `T` 实现了 `A` 或 `B`”）难以直接表达，限制了泛型代码的灵活性。

- **C++20 的 Concepts** 虽然在语法上对模板约束进行了改进，但其本质仍未脱离“鸭子类型”的范畴。它主要检查语法层面的合法性（例如，表达式 `a + b` 是否可以被解析），而无法深入验证更复杂的语义约束。深层次的类型错误依然需要等待模板被完全实例化后才能被发现，导致了其著名的冗长且难以理解的错误信息。

1.1.3 本文贡献：命题式类型约束 (PTC)

现有方案的局限性表明，我们需要一种新的范式来平衡静态分析的“不可能三角”。本文提出并设计了一种名为**命题式类型约束 (Propositional Type Constraints, PTC)**的统一框架。

PTC 的核心思想是将编程语言的类型约束系统，从一个处理特定规则（如 Trait Bound）的子系统，提升为一个能够理解和推理**一阶逻辑命题**的通用静态分析引擎。它将类型检查过程本身，重塑为一个逻辑证明过程，旨在统一并超越现有模型的表达力，同时保持甚至增强定义期的可检查性。

1.2 命题式类型约束的形式化定义

1.2.1 1. 语法 (Syntax)

我们首先定义构成命题式类型约束系统的各个语法类别。

1.2.1.1 基础类别

- **类型 (Types)**

$$
\begin{aligned}
T, U, V ::= & \text{Int} \mid \text{Bool} \mid \ldots && \text{Basic Types} \\
& \mid \alpha, \beta, \gamma && \text{Type Variables} \\
& \mid C(T_1, \ldots, T_n) && \text{Type Constructor Application} \\
& \mid \ldots
\end{aligned}
$$

- **项 (Terms)**

$$
\begin{aligned}
t, e, v ::= & x, y, z && \text{Term Variables} \\
& \mid c && \text{Constants, e.g., } 1, \text{true} \\
& \mid f(t_1, \ldots, t_n) && \text{Function Application} \\
& \mid \ldots
\end{aligned}
$$

- **Trait (特征)**

$$
A, B, C ::= \text{Display} \mid \text{Add} \mid \text{Eq} \mid \ldots \quad \text{Trait Identifiers}
$$

1.2.1.2 命题 (Propositions)

命题 `p`，`q` 是我们逻辑系统中的核心断言。

- **原子命题 (Atomic Propositions)**

$$
\begin{aligned}
p_{\text{atomic}} ::= & t : T && \text{Type Assertion: Term } t \text{ has type } T \\
& \mid T \text{ :- } A && \text{Type Trait Assertion: Type } T \text{ implements trait } A \\
& \mid t \text{ :- } A && \text{Term Trait Assertion: Term } t \text{ 's type implements trait } A \\
& \mid T_1 = T_2 && \text{Type Equality: Type } T_1 \text{ equals } T_2 \\
& \mid t_1 = t_2 && \text{Term Equality: Term } t_1 \text{ equals } t_2 \\
& \mid \top && \text{True} \\
& \mid \bot && \text{False}
\end{aligned}
$$

- **复合命题 (Compound Propositions)**

$$
\begin{aligned}
p, q ::= & p_{\text{atomic}} && \text{Atomic Propositions} \\
& \mid \neg p && \text{Negation: not } p \\
& \mid p \wedge q && \text{Conjunction:} p \text{ and } q \\
& \mid p \vee q && \text{Disjunction:} p \text{ or } q \\
& \mid p \Rightarrow q && \text{Implication:} p \text{ implies } q
\end{aligned}
$$

1.2.1.3 上下文与判断 (Contexts and Judgements)

- **上下文 (Context)**: 上下文 Γ 是一个由类型断言和命题断言构成的集合,用于在类型检查和推理中提供已知的事实。

$$
\Gamma ::= \emptyset \mid \Gamma, p
$$

- **判断 (Judgement)**: 类型系统的核心判断形式为 $\Gamma \vdash p$,读作"在上下文 Γ 中,命题 p 为真"。

1.2.2 语义 (Semantics)

我们非形式化地描述这些构造的含义。

- **$t : T$ (类型断言)**: 这个命题为真,如果根据语言的类型规则,可以推导出项 `t` 的类型是 `T`。这是传统类型检查的核心。

- T :- A **(类型特征断言)**: 这个命题为真,如果存在一个该语言中合法的 `impl A for T` 声明。这是一个关于**全局实现**的断言。
- t :- A **(项特征断言)**: 这是 T :- A 的一种语法糖。它为真,如果存在类型 `T` 使得 $\Gamma \vdash t : T$ 并且 $\Gamma \vdash T$:- A。它断言一个**具体的值**满足某个特征,通常用于需要动态分派或在运行时检查的场景,但在静态分析中,它等价于对其类型的断言。
- $T_1 = T_2$ **(类型相等)**: 这个命题为真,如果类型 T_1 和 T_2 在当前类型系统的规则下是等价的(例如,通过别名、类型计算或合一)。
- $t_1 = t_2$ **(项相等)**: 这个命题为真,如果项 t_1 和 t_2 在**编译期**可以被证明指向同一个值。这通常只对编译期常量或可被符号执行的纯函数有意义。
- **逻辑连接词 ($\neg, \wedge, \vee, \Rightarrow$)**: 它们具有标准的一阶逻辑语义。
  - $\Gamma \vdash \neg p$ 为真,当且仅当 $\Gamma \vdash p$ 不为真。
  - $\Gamma \vdash p \wedge q$ 为真,当且仅当 $\Gamma \vdash p$ 和 $\Gamma \vdash q$ 都为真。
  - $\Gamma \vdash p \vee q$ 为真,当且仅当 $\Gamma \vdash p$ 或 $\Gamma \vdash q$ 至少一个为真。
  - $\Gamma \vdash p \Rightarrow q$ 为真,当且仅当"若 $\Gamma \vdash p$ 为真,则 $\Gamma \vdash q$ 也为真"。

1.2.3 在编程语言构造中的示例

命题式类型约束通过 `where` 子句、`requires` 和 `ensures` 等语言构造,被整合到编程语言中。

- **泛型函数声明**:

$$\text{fn } f\langle\alpha_1, ...\rangle(x_1 : T_1, ...) \to U \text{ where } p$$

  这里的 `p` 是一个关于类型变量 α_i 和参数 x_i 的命题。它构成了该函数签名的**契约 (Contract)**。

- **函数体类型检查**: 对于上述函数 `f`,其函数体的类型检查判断为:

$$\Gamma, x_1 : T_1, ..., p \vdash \text{body} : U' \text{ 并且 } \Gamma, x_1 : T_1, ..., p \vdash U' = U$$

  即,在将函数签名中的**命题 `p` 作为假设加入上下文**后,函数体必须是类型正确的。

- **函数调用检查**: 对于调用 `f<V₁,...>(e₁,...)`,编译器需要证明:

$$\Gamma \vdash p[\alpha_1 \mapsto V_1, ..., x_1 \mapsto e_1, ...]$$

  即,在当前的调用点上下文 Γ 中,将泛型参数和实际参数代入到函数签名的命题 `p` 后,得到的**新命题必须为真**。

1.3 PTC 的表达力：案例研究

PTC 作为一个更通用的框架，其表达能力远超传统的 Trait 系统。本章将通过一系列案例，展示 PTC 如何优雅地表达复杂的类型关系。

1.3.1 模拟与泛化现有系统

PTC 首先可以完美地描述并泛化现有的约束系统。

1.3.1.1 描述 Trait 系统

Rust 的 Trait 系统可以被 PTC 的子集同构地描述。例如，一个泛型函数签名：

```
fn sort<T: Ord>(slice: &mut [T])
```

可以被 PTC 的蕴含关系精确表达。其核心逻辑规则为 `∀T. (T :- Ord) ⇒ (sort<T> is well-typed)`。这证明了 PTC 是对霍恩子句约束系统的一种自然泛化。

1.3.1.2 构造联合类型

传统静态类型语言通常难以处理异构集合或需要多种输入类型的函数。PTC 通过逻辑析取 (`∨`) 和类型相等 (`=`)，可以静态地、安全地表达联合类型约束。

```
fn process(input: T) where T = String ∨ T = i32 {
  // ...
}
```

此约束精确地限定了 `process` 函数只接受 `String` 或 `i32` 类型的参数。编译器可以在调用点验证这一约束，并在函数体内部利用此信息进行类型精化。

1.3.2 更多可能拓展原子命题

PTC 的真正威力在于其可扩展的原子命题集合，允许对类型的更多维度进行断言。

1.3.2.1 结构化断言

PTC 允许对值的**内部结构**进行静态断言，这是传统 Trait 系统无法做到的。

```
-- 命题：项 `t` 拥有一个名为 `to_string` 的方法
t :~ to_string: fn(*itself) -> String

-- 命题：类型 `T` 拥有一个名为 `data` 的、类型为 `u32` 的字段
T :~ data: u32
```

这些“结构化断言”类似于行多态（Row Polymorphism）或结构化类型，允许编写能够操作具有特定“形状”而非特定名义类型的泛型代码，极大地增强了代码的通用性。

1.4 发生类型：PTC 指导下的上下文精化

PTC 的强大表达力，在与**发生类型 (Occurrence Typing)**机制结合时，得到了最大程度的发挥。发生类型允许编译器根据代码路径上的条件，在不同的分支中“精化”对类型的认知。Flurry 通过 `inline if` / `inline when` 等编译期控制流构造来实现这一点。

1.4.1 `inline` 控制流的符号执行

当编译器遇到 `inline if p { ... }` 时，它会对条件 `p`（一个 PTC 命题）进行符号执行。

- 在 `then` 分支中，编译器将命题 `p` 加入到当前的**路径条件 (Path Condition)**中。
- 在 `else` 分支中，则将 `¬p` 加入路径条件。

路径条件是一个逻辑上下文，它精确地记录了在当前代码点所有已知为真的事实。

1.4.2 精化类型环境

这个被增强的路径条件，会极大地影响分支内部的类型检查行为，即**精化类型环境**。

- **约束精化**:

  ```
  fn process(t: T) where T {
    inline if T :- Display {
      -- 在此作用域内，编译器知道 `T` 实现了 `Display`
      -- 因此，对 `t` 调用 Display 相关的方法是合法的
    }
  }
  ```

- **类型精化**:

```
fn process(t: T) where T, requires T == String or T == i32 {
  inline if T == String {
    -- 在此作用域内，泛型 `T` 不再是未知的
    -- 它被精确地精化为 `String` 类型
    -- 因此，可以安全地调用 String 的方法
  } else {
    -- 在此作用-域内，通过逻辑推导 (A ∨ B) ∧ ¬A ⇒ B
    -- 编译器知道 `T` 必然是 `i32`
  }
}
```

通过这种 PTC 指导下的上下文精化，Flurry 允许开发者编写出能够安全、高效地处理多种类型和复杂条件的泛型代码，其代码特化能力和静态分析精度远超现有语言。

2 要求与数据

2.1 基本理论要求

1. 具备扎实的计算机科学基础，熟悉数据结构与算法，掌握一定的编程能力。
2. 具备一定的英语阅读能力，能够理解相关领域的文献资料。
3. 具备编译工程能力，包括词法分析、语法分析、语义分析和代码生成等编译器构造的核心技术。
4. 掌握 CDCL(T) SMT solver 构造能力，理解冲突驱动子句学习算法和理论求解器的集成方法。
5. 具备 SLG 框架构造能力，熟悉选择线性确定性(Selective Linear Definite)子句解析和表格化方法。
6. 掌握泛型类型推导设计能力，理解类型变量统一、约束求解和类型实例化等核心算法。
7. 具备上下文类型推导设计能力，熟悉类型环境管理、作用域分析和类型精化技术。
8. 掌握符号执行理论，理解路径条件管理、约束求解和程序状态空间探索等关键概念。

2.2 系统具体要求

1. 设计并实现一个支持命题式类型约束的编程语言，具备基本的语法和语义。
2. 实现一个编译器，能够解析、类型检查并生成中间代码。
3. 集成一个 CDCL(T) SMT solver，用于处理复杂的类型约束和逻辑推理。
4. 实现一个 SLG 框架，用于支持一阶逻辑的霍恩子句子集。

3 应完成的工作

1. 毕业设计开题报告 1 份；
2. 英文翻译材料 1 份（包括不少于 2 万字符的英文原文和译文）；
3. 完成相关软件一套，给出程序清单，用户使用说明书；
4. 毕业设计说明书 1 份（不少于 1.5 万字，附中英文摘要，其中英文摘要 300～500 个英文单词）。

4 应收集的资料及主要参考文献

参考文献

[1] D. Racordon 和 B. Chung,《Use Site Checking Considered Harmful》, 收入 *Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 收入 Onward! '24.

Pasadena, CA, USA: Association for Computing Machinery, 2024, 页 275–287. doi: 10.1145/3689492.3689814.

[2] A. Kovács,《Staged compilation with two-level type theory》, *Proc. ACM Program. Lang.*, 卷 6, 期 ICFP, 8 月 2022, doi: 10.1145/3547641.

[3] G. Castagna, M. Laurent, K. Nguyễn, 和 M. Lutze,《On type-cases, union elimination, and occurrence typing》, *Proc. ACM Program. Lang.*, 卷 6, 期 POPL, 1 月 2022, doi: 10.1145/3498674.

[4] L. de Moura 和 N. Bjørner,《Z3: An Efficient SMT Solver》, 收入 *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan 和 J. Rehof, 编, Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, 页 337–340.

[5] W. Chen 和 D. S. Warren,《Query evaluation under the well-founded semantics》, 收入 *Proceedings of the Twelfth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*, 收入 PODS '93. Washington, D.C., USA: Association for Computing Machinery, 1993, 页 168–179. doi: 10.1145/153850.153865.

[6] F. Barbanera, M. Dezaniciancaglini, 和 U. Deliguoro,《Intersection and Union Types: Syntax and Semantics》, *Information and Computation*, 卷 119, 期 2, 页 202–230, 1995, doi: https://doi.org/10.1006/inco.1995.1086.

[7] R. Team,《Chalk Book》. [在线]. 载于: https://rust-lang.github.io/chalk/book/

[8] M. Odersky, M. Sulzmann, 和 M. Wehr,《Type inference with constrained types》, *Theory and Practice of Object Systems*, 卷 5, 期 1, 页 35–55, 1999, doi: https://doi.org/10.1002/(SICI)1096-9942(199901/03)5:1<35::AID-TAPO4>3.0.CO;2-4.

[9] B. Demoen, M. J. Garcia De La Banda, 和 P. J. Stuckey, *Type Constraint Solving for Parametric and Ad-hoc Polymorphism*. Australia: Monash University, 1999.

[10] D. Orchard, V.-B. Liepelt, 和 H. Eades III,《Quantitative program reasoning with graded modal types》, *Proc. ACM Program. Lang.*, 卷 3, 期 ICFP, 7 月 2019, doi: 10.1145/3341714.

[11] J. Dunfield 和 N. Krishnaswami,《Bidirectional Typing》, *ACM Comput. Surv.*, 卷 54, 期 5, 5 月 2021, doi: 10.1145/3450952.

[12] X. Xue 和 B. C. d. S. Oliveira,《Contextual Typing》, *Proc. ACM Program. Lang.*, 卷 8, 期 ICFP, 8 月 2024, doi: 10.1145/3674655.

[13] M. Hassan, C. Urban, M. Eilers, 和 P. Müller,《MaxSMT-Based Type Inference for Python 3》, 收入 *Computer Aided Verification*, H. Chockler 和 G. Weissenbacher, 编, Cham: Springer International Publishing, 2018, 页 12–19.

[14] L. Parreaux, A. Boruch-Gruszecki, A. Fan, 和 C. Y. Chau,《When Subtyping Constraints Liberate: A Novel Type Inference Approach for First-Class Polymorphism》, *Proc. ACM Program. Lang.*, 卷 8, 期 POPL, 1 月 2024, doi: 10.1145/3632890.

[15] A. Fan, H. Xu, 和 N. Xie,《Practical Type Inference with Levels》, *Proc. ACM Program. Lang.*, 卷 9, 期 PLDI, 6 月 2025, doi: 10.1145/3729338.

5 试验、测试、试制加工所需主要仪器设备

操作系统： Arch Linux 开发工具：Vscode, Git, Cargo, Rust Analyzer, sbt 版本管理平台：GitHub

6 毕业设计开始与完成时间

2025-2026 学年第一学期第 8 18 周，2025-2026 学年第二学期第 1 11 周